Presentation Agenda

A structured journey through every major type of software testing

01 **Introduction to Testing**

Definitions • Goals • SDLC fit • Overview

02 **Broad Categories**

Functional • Non-Functional • Manual • Automated

03 **Functional Testing Types**

Unit • Integration • System • UAT • Smoke • Sanity • Regression

04 **Non-Functional Testing**

Performance • Security • Usability • Compatibility • Reliability • Scalability

05 **Manual vs Automated**

Comparison • Advantages • When to use each

06 **Exploratory & Ad-hoc**

Definitions • Benefits • Use cases

07 **Alpha & Beta Testing**

Internal vs external • Real-world examples

08 **Test Automation in QA**

Selenium • Playwright • IntelliJ • Eclipse • Workflow

09 **Real-World Workflow**

10 **Best Practices & Summary**

SECTION

1

Introduction to Software Testing

What testing is, why it matters, and where it fits in the delivery lifecycle

What is Software Testing?

The formal discipline that ensures software meets its intended purpose

Software Testing is the systematic process of executing a software system or component to EVALUATE whether it behaves as expected, meets specified requirements, and is free from defects. It answers three core questions: Does it DO what it should? Does it NOT do what it shouldn't? Is it good enough to release?

🎯 Goals of Software Testing

- Quality — verify the software meets functional requirements
- Reliability — ensure consistent, repeatable behaviour
- Performance — confirm the system handles expected load
- Security — protect against vulnerabilities and breaches
- Usability — validate the experience for the end user
- Compliance — meet industry standards and regulations

❌ What Happens Without Testing

- Defects reach production — real users are affected
- Security vulnerabilities expose sensitive user data
- System crashes under load lose revenue and trust
- Regulatory fines for non-compliant software
- Reputation damage — one viral bug can end a product
- Cost of fixing production bugs is 100× more expensive

Where Testing Fits — SDLC & STLC

Testing is not a phase at the end — it runs throughout the entire delivery lifecycle

SDLC — Software Development Life Cycle

QA ACTIVITIES AT EACH PHASE

Requirements	Design	Development	Testing	Deployment	Maintenance
Review requirements for testability	Create Test Plan	Write test cases	Execute all test types	Smoke test post-deploy	Monitor & regression test

🔑 Shift-Left Testing: Modern QA starts in the Requirements phase — not after coding. Finding a defect in requirements costs 1× to fix. Finding it in production costs 100×.

TESTING TYPES THAT APPLY PER PHASE

Requirements:	Requirement review, RTM, UAT planning
Development:	Unit Testing, Code review, Static analysis
Testing:	Integration, System, Performance, Security, Regression, UAT
Deployment:	Smoke Testing, Sanity Testing
Maintenance:	Regression, Monitoring, Ad-hoc, Exploratory

Overview of Testing Categories

The complete landscape — every type of software testing at a glance

Functional Testing

- Unit Testing
- Integration Testing
- System Testing
- UAT
- Smoke Testing
- Sanity Testing
- Regression Testing

Non-Functional Testing

- Performance Testing
- Load Testing
- Stress Testing
- Security Testing
- Usability Testing
- Compatibility Testing
- Scalability Testing

Manual Testing

- Exploratory Testing
- Ad-hoc Testing
- UAT
- Usability Testing
- Alpha Testing
- Checklist-based
- Scenario Testing

Automated Testing

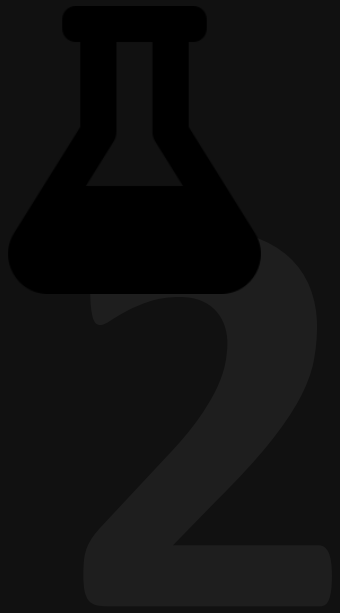
- Unit Test Automation
- Regression Automation
- API Testing
- Performance Testing
- CI/CD Pipeline Tests
- Cross-browser Testing
- Data-Driven Testing

SECTION

2

Broad Categories of Testing

Understanding the four fundamental ways to classify software testing



Functional vs Non-Functional Testing

The most fundamental split in all of software testing

⚙ Functional Testing — WHAT it does

- Tests whether specific FUNCTIONS of the software work as specified in requirements
- Focuses on business logic, user actions, and expected outputs
- Each test has a clear input, execution step, and expected output
- Pass/Fail is determined by functional requirements
- Examples: Can the user log in? Does the cart calculate correctly?
- Types: Unit, Integration, System, UAT, Smoke, Sanity, Regression
- Performed by: QA engineers, developers (unit tests), business users (UAT)

📊 Non-Functional Testing — HOW WELL it does it

- Tests the QUALITY ATTRIBUTES of the system, not specific functions
- Focuses on performance, security, reliability, usability, and scalability
- Tests system behaviour under conditions rather than specific feature outcomes
- Often requires specialised tools and expertise
- Examples: Does it load in under 3 seconds? Can it handle 10,000 users?
- Types: Performance, Load, Stress, Security, Usability, Compatibility
- Performed by: Specialised QA engineers, DevOps, security teams

Manual vs Automated Testing

Two execution approaches — each with distinct strengths and purposes

Manual Testing

- A human tester executes test cases step by step without automation
- Relies on human observation, judgement, and experience
- Excellent for exploratory, usability, and one-off tests
- Flexible — can adapt test direction based on what is observed
- No scripting or programming skill required to execute
- Slower at scale and prone to human inconsistency
- Best for: New features, UI/UX evaluation, edge case exploration

Automated Testing

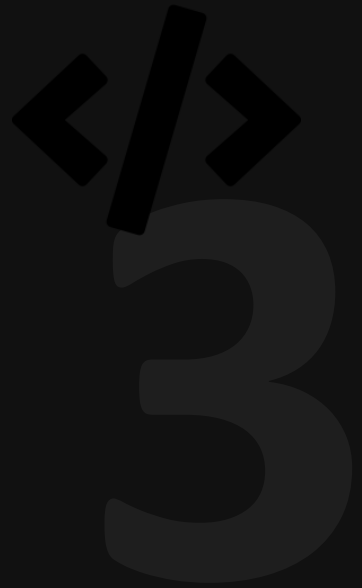
- Software scripts execute test cases automatically without human interaction
- Scripts use frameworks like Selenium (Java/Eclipse/IntelliJ) or Playwright
- Extremely fast at scale — 1,000 tests in minutes vs days manually
- Consistent and repeatable — eliminates human error in execution
- Requires coding skill and ongoing script maintenance
- Not suitable for subjective or exploratory test scenarios
- Best for: Regression suites, CI/CD pipelines, repetitive flows

SECTION

3

Functional Testing Types

Unit • Integration • System • UAT • Smoke • Sanity • Regression — in full detail



Unit Testing

Testing the smallest individual units of code in complete isolation

Functional

Unit Testing is the testing of a single, isolated unit of code — typically a function, method, or class — to verify it produces the correct output for a given input. Units are tested in complete isolation from dependencies (databases, APIs) using 'mocks' or 'stubs'.

Key Characteristics

- Performed by DEVELOPERS — not QA engineers (though QA reviews)
- Written alongside the code (Test-Driven Development = TDD)
- Each unit test covers ONE specific behaviour or code path
- Fast execution — 1,000 unit tests run in seconds
- Isolated: all external dependencies are mocked/stubbed
- First layer in the Testing Pyramid — the broadest and cheapest
- Catches logic errors at the source, before integration

Industry Tools

- JUnit 5 — Java unit testing framework (Eclipse & IntelliJ)
- TestNG — Alternative Java framework with rich annotations
- Mockito — Java mocking framework for isolating dependencies
- Jest — JavaScript unit testing framework
- NUnit — .NET/C# unit testing framework

Real-World Example

- Function: calculateDiscount(price, discountPct)
- Unit Test 1: calculateDiscount(100, 10) → expects 90 ✅
- Unit Test 2: calculateDiscount(0, 10) → expects 0 ✅
- Unit Test 3: calculateDiscount(100, -5) → expects error ✅
- Unit Test 4: calculateDiscount(null, 10) → expects error ✅

Integration Testing

Verifying that components work correctly when combined together

Functional

Integration Testing verifies that MULTIPLE units or components work correctly when they are combined. While unit tests check individual pieces in isolation, integration testing checks the 'joints' — the interfaces, data flows, and communication between components.

↓ Top-Down Integration

Testing begins from the TOP-LEVEL modules downward. Higher-level modules are tested first, with lower-level modules replaced by 'stubs' (simulated components) until the real ones are integrated. Advantage: High-level business logic is validated early.

↑ Bottom-Up Integration

Testing begins from the LOWEST-LEVEL modules upward. Lower-level components are tested first, with higher-level modules replaced by 'drivers' (test harnesses). Advantage: Core utilities and database layers are validated thoroughly.

Hybrid Sandwich

Combines both approaches simultaneously. Top-down for business logic, bottom-up for data layers. Most common in large enterprise projects. Advantage: Parallel testing reduces time-to-integrate. Most realistic but most complex

Tools

- Selenium / Playwright — UI-level integration flows
- Postman / RestAssured — API integration verification
- MockServer / WireMock — mock external services

Real-World Example

- E-commerce: User adds to cart (UI) → Cart service (API) → Inventory DB updated
- Login module integrates with Auth service integrates with User DB

System Testing

End-to-end validation of the complete, integrated software system

Functional

System Testing validates the COMPLETE, INTEGRATED system against its specified requirements. Unlike integration testing (which tests component interfaces), system testing treats the entire application as a black box — testing it exactly as a real user would, end to end.

What System Testing Covers

- Complete end-to-end business workflows and user journeys
- All functional requirements specified in the BRD/FSD
- Interactions between ALL system components simultaneously
- Error handling, boundary conditions, and negative paths
- Data persistence across multiple sessions and actions
- System-level security and access control
- All GUI elements, forms, and user interaction flows

Approach & Tools

- Black-box approach — tester has no visibility of code
- Selenium (Eclipse/IntelliJ) — automated UI workflow testing
- Playwright — cross-browser end-to-end system tests
- TestRail / Zephyr — test case management and tracking
- Executed by the QA engineering team in the QA environment

Real-World Scenario

- Banking app system test flow:
- 1. Register new account → 2. Verify email → 3. Log in
- 4. Transfer £500 between accounts → 5. Confirm transaction history
- 6. Download statement as PDF → 7. Log out
- Every step in this flow is verified with real or realistic test data

User Acceptance Testing (UAT)

Business stakeholders validate the software meets their needs before go-live

Functional

UAT is the final formal testing phase, performed by BUSINESS USERS and stakeholders — not QA engineers. Its purpose is to confirm that the software meets the business requirements and is acceptable for release. It is the user's official sign-off that the software is fit for purpose.

Who Performs UAT

- Business users / end users who will use the system daily
- Product owners and business analysts (BA team)
- Client representatives and key stakeholders
- Sometimes external beta testers (see Beta Testing)
- QA engineers SUPPORT UAT but do not perform it

UAT Activities

- Review and approve UAT test scenarios (written from business perspective)
- Execute test cases using their own knowledge of business processes
- Compare results against business expectations (not technical specs)
- Raise issues using the agreed defect management process
- Formally sign off or reject the software for release

TYPES OF UAT

Alpha UAT

Internal business users test in a controlled environment before external release. Company employees or selected customers.

Beta UAT

External real users test in a near-production environment. Software is released to a limited group of actual end users.

Contract Acceptance

Software is tested against the criteria defined in the client contract. Client formally accepts or rejects delivery.

Smoke Testing

A quick health check to verify the build is stable enough for full testing

Functional

Smoke Testing (also called 'Build Verification Testing') is a shallow, fast test suite that verifies the MOST CRITICAL functions of a new build work before full testing begins. The name comes from hardware testing: power on a circuit and see if it smokes — if it does, stop immediately.

What Smoke Testing Checks

- Application launches and the home/login page loads correctly
- Authentication: login and logout functions are accessible
- Core navigation paths are available and responsive
- Critical database connections are functioning
- Key API endpoints return expected HTTP status codes
- No blockers that would prevent any testing from proceeding

Key Characteristics

- Typically 10–20 test cases — NOT comprehensive
- Executed FIRST, before any other testing begins
- Takes 15–30 minutes to complete (often automated)
- PASS → Proceed with full testing cycle
- FAIL → STOP all testing, escalate to DevOps/Dev team
- Run automatically in CI/CD pipelines after every deployment

Tools for Smoke Testing

- Playwright / Selenium — automated smoke scripts triggered by CI
- Postman — API smoke test collections via Newman runner
- Jenkins / GitHub Actions — triggers smoke on every build deploy

Real-World Example

- E-commerce post-deployment smoke tests:
- 1. Homepage loads (< 3s) 2. Login works 3. Product search returns results
- 4. Add to cart works 5. Payment gateway reachable
- If any of these 5 checks fail → deployment is rolled back

Sanity Testing

Targeted verification that a specific fix or change works correctly

Functional

Sanity Testing is a NARROW, FOCUSED check performed after a minor code change or bug fix to verify that the specific change works correctly and has not broken any closely related functionality. Unlike smoke testing (which covers the whole system broadly), sanity testing covers one area in depth.

SMOKE TESTING vs SANITY TESTING — COMPARISON

Aspect	Smoke Testing	Sanity Testing
Scope	Broad — covers entire application	Narrow — covers specific changed area
Depth	Shallow — surface-level checks only	Deep — thorough within a focused area
Purpose	Is the build STABLE enough to test?	Does this SPECIFIC CHANGE work correctly?
When used	After every new build is deployed	After a bug fix or minor change
Who performs	QA team (often automated)	QA engineer (usually manual)
Documented?	Always documented and tracked	Often undocumented (subset of test suite)
Duration	15–30 minutes (10–20 tests)	30–60 minutes (focused area)

💡 *Example: Developer fixes a bug where the 'Apply Coupon' field accepts negative values. Sanity testing checks: (1) negative values are now rejected, (2) valid coupons still work, (3) cart total still calculates correctly.*

Regression Testing

Ensuring new changes haven't broken existing working functionality

Functional

Regression Testing re-executes a subset (or all) of existing test cases after ANY code change — bug fixes, new features, or configuration updates — to verify that previously working functionality has NOT been accidentally broken. It is the safety net of software quality.

Why Regression Testing is Critical

- Every code change creates risk — developers can inadvertently break things
- Complex systems have unexpected dependencies between components
- Without regression, defects 'regress' — old bugs reappear
- Stakeholders need confidence that changes don't break existing features
- Required before every release to production environment
- Critical in Agile — features added every sprint must not break previous sprints

Automation for Regression

- Regression is the PRIMARY use case for test automation
- Selenium (Java/IntelliJ/Eclipse) — web app regression suites
- Playwright — fast, reliable cross-browser regression execution
- TestNG / JUnit — organise regression tests into suites
- CI/CD integration: regression runs automatically on every merge
- 500 regression tests in 20 minutes vs 3 days manually

REGRESSION TESTING STRATEGIES

Full Regression

All test cases re-run. Used before major releases. Most thorough but most time-consuming.

Partial Regression

Subset of tests related to the changed area. Used for minor changes. Faster but less comprehensive.

Selective Regression

Only tests mapped to changed requirements via RTM. Requires strong traceability. Most efficient.

SECTION

4

Non-Functional Testing Types

Performance • Security • Usability • Compatibility • Reliability • Scalability

Performance Testing — Overview

Testing HOW WELL the system performs under various conditions

Non-Functional

Performance Testing evaluates the system's responsiveness, stability, speed, and scalability under various workload conditions. It answers: How fast does it respond? How many users can it handle? When does it break? How does it behave over extended periods?

📈 Load Testing

Simulates expected normal user load. Validates system performs acceptably under anticipated traffic. Goal: Confirm response times and resource usage within acceptable limits at peak expected load.

💥 Stress Testing

Pushes the system BEYOND its limits to find the breaking point. Validates how the system fails and whether it recovers gracefully. Goal: Find the maximum capacity and validate failure behaviour.

⚡ Spike Testing

Sudden, extreme increases in load for a short period. Simulates viral traffic events (flash sales, breaking news). Goal: Validate system response to sudden traffic surges without crashing.

⏳ Endurance (Soak) Testing

Runs the system under sustained normal load for an extended period (hours/days). Identifies memory leaks and resource degradation over time. Goal: Confirm long-term stability and no gradual performance decay.

Performance Testing — Tools & Real-World Application

Industry tools and practical examples for each performance test type

Non-Functional

Type	Scenario Tested	Example	Tool	Pass Criterion
Load	10,000 concurrent users checkout simultaneously	Black Friday sale on e-commerce platform	JMeter, Gatling, k6	< 3s response time at peak load
Stress	150% of maximum expected users	Traffic spike after viral social media post	JMeter, Locust	Graceful degradation, auto-recovery
Spike	0 to 50,000 users in 30 seconds	New iPhone announced — Apple.com traffic	Gatling, k6	No crash; queue or CDN handles overflow
Endurance	5,000 users for 72 hours continuous	Online banking system daily operations	JMeter long runs	No memory leak; response time stable

PERFORMANCE METRICS MEASURED

Response Time

Time from user action to system response. Target: < 2s for web pages, < 100ms for APIs

Throughput

Requests processed per second. Indicates system processing capacity at scale

Error Rate

Percentage of failed requests. Acceptable threshold: typically < 1% under load

CPU / Memory

Resource utilisation during load. High CPU (>85%) or memory leaks indicate inefficiency

Security Testing

Identifying vulnerabilities before attackers do

Non-Functional

Security Testing evaluates the software system's ability to protect data, maintain functionality, and resist malicious attacks. It aims to find vulnerabilities, weaknesses, and risks in the application before they can be exploited by real attackers.

Vulnerability Testing

- Systematic scanning of the application for known security weaknesses
- Checks for OWASP Top 10: SQL injection, XSS, broken auth, insecure API
- Uses automated scanners to identify vulnerabilities at scale
- Tools: OWASP ZAP, Burp Suite, Nessus, SonarQube (static analysis)
- Example: Scanning login form for SQL injection vulnerabilities

Penetration Testing (Pen Testing)

- Ethical hackers attempt to actively exploit vulnerabilities
- Simulates a real-world attacker — goes beyond automated scanning
- Identifies complex, chained vulnerabilities that scanners miss
- Performed by certified ethical hackers (CEH, OSCP certifications)
- Example: Manually attempting to bypass authentication and access admin data

OWASP TOP 10 — THE MOST CRITICAL SECURITY RISKS

1. Broken Access Control	2. Cryptographic Failures	3. Injection (SQL/Command/LDAP)	4. Insecure Design	5. Security Misconfiguration
6. Vulnerable Components	7. Auth & Identity Failures	8. Software Integrity Failures	9. Logging Failures	10. Server-Side Request Forgery

Usability Testing

Evaluating how easy and intuitive the software is to use

Non-Functional

Usability Testing evaluates the software from the USER'S PERSPECTIVE — assessing ease of use, intuitiveness, accessibility, and user satisfaction. The focus is on the human experience rather than technical correctness. Software can be functionally correct but still be unusable.

What Usability Testing Measures

- Effectiveness — can users complete tasks successfully?
- Efficiency — how long does it take to complete tasks?
- Learnability — how quickly can new users learn the system?
- Memorability — how easily can users remember how to use it?
- Error rate — how often do users make mistakes?
- Satisfaction — how pleasant is the overall experience?
- Accessibility — can users with disabilities use the system?

How Usability Testing Is Done

- Recruit representative real users (5–8 participants per round)
- Give users tasks to complete WITHOUT assistance
- Observe and record: where they hesitate, click wrong, or struggle
- Think-aloud protocol: users verbalise their thoughts as they navigate
- Record sessions using tools: Hotjar, Lookback, UserTesting.com
- Analyse heatmaps, click maps, and session recordings
- Iterate design based on findings — test again after changes

Real Example: Amazon simplified its checkout to One-Click Ordering after usability testing revealed that multi-step checkout caused 70% cart abandonment

Compatibility Testing

Ensuring the software works across all target browsers, devices, and platforms

Non-Functional

Compatibility Testing verifies that the software works correctly across DIFFERENT browsers, operating systems, devices, screen resolutions, and network conditions. With users accessing software on thousands of device and browser combinations, compatibility is critical to reach all intended users.

Cross-Browser Testing

- Verify identical behaviour across all target web browsers
- Chrome, Firefox, Safari, Edge, Opera — all have different rendering engines
- Same HTML/CSS/JavaScript behaves differently per browser
- Test all browser versions specified in browser matrix
- Playwright: natively runs Chromium, Firefox, WebKit (Safari) in ONE test
- Selenium Grid: run tests in parallel across multiple browsers
- BrowserStack / Sauce Labs: cloud-based real browser testing at scale
- Example: CSS grid layouts that render correctly in Chrome but break in Safari

Cross-Device Testing

- Verify behaviour on desktop, tablet, and mobile devices
- Different screen sizes, resolutions, and pixel densities
- Touch vs mouse input — different interaction patterns
- Mobile operating systems: iOS (Safari) and Android (Chrome)
- Responsive design breakpoints must be tested at each screen size
- Device emulation in Chrome DevTools for development testing
- Real device testing required for final validation (not just emulators)
- Example: Navigation menu collapses correctly on 375px mobile viewport

Key Tools: Playwright (built-in cross-browser) • Selenium Grid (parallel browsers) • BrowserStack (real devices) • Sauce Labs (cloud) • LambdaTest (browser cloud) •

Reliability & Scalability Testing

Stability over time and the ability to grow with demand

Non-Functional

RELIABILITY TESTING

Definition: Reliability Testing assesses the system's ability to PERFORM CONSISTENTLY and WITHOUT FAILURE over a specified period and under defined conditions. It measures the probability that the system will work without errors for a given time period.

Reliability Metric	Definition	Example	Target
MTBF (Mean Time Between Failures)	Average time between system failures	Banking app fails once every 2,000 hours	MTBF > 1,000 hours
MTTR (Mean Time To Recover)	Average time to restore after a failure	Payment service recovery: 4 minutes	MTTR < 15 minutes
Availability	Percentage of time system is operational	99.9% = 8.7 hours downtime/year	≥ 99.9% ('three nines')

SCALABILITY TESTING

Definition: Scalability Testing verifies that the system can HANDLE GROWTH — in users, data volume, transactions, or geographic distribution — without performance degradation. It tests both horizontal (adding more servers) and vertical (increasing server capacity) scaling.

User Scalability

Test with 100 → 1K → 10K → 100K users. At what point does performance degrade?

Data Scalability

Database grows from 10K to 100M records. Does query time remain acceptable?

Geographic Scaling

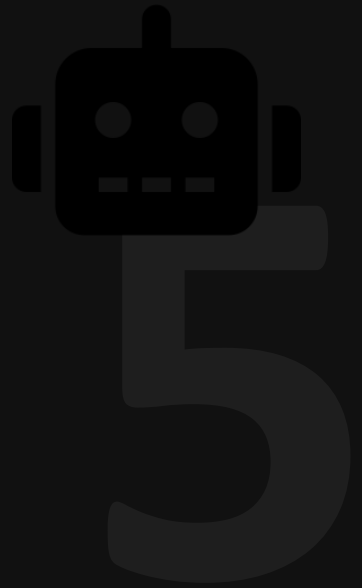
Users from 5 countries simultaneously. CDN and latency tested globally.

SECTION

5

Manual vs Automated Testing

A deep comparison — when to use each approach for maximum QA effectiveness



Manual vs Automated Testing — Detailed Comparison

Choosing the right approach for each testing scenario

Factor	Manual Testing	Automated Testing
Execution	Human tester follows steps manually	Script executes steps automatically
Speed	Slow — 1 test at a time	Fast — 1,000s of tests in parallel
Accuracy	Prone to human error/fatigue	Consistent — exact same steps every run
Cost (Initial)	Low — no scripting investment	High — requires scripting time upfront
Cost (Ongoing)	High — scales linearly with test count	Low — same script runs infinitely
Maintenance	None — test case documents updated	Scripts need updating when UI changes
Flexibility	High — can adapt during execution	Low — follows the script exactly
Best For	Exploratory, usability, one-off, UAT	Regression, CI/CD, data-driven, performance
Skill Required	Testing knowledge, domain expertise	Programming + testing + tooling knowledge

✅ *Use Manual When: New/unstable features • Exploratory • UX review • Ad-hoc • UAT*

🤖 *Use Automation When: Regression • CI/CD • High-frequency • Performance • Cross-browser*

Manual vs Automated — When To Use Each

Real-world decision framework with practical examples

AUTOMATE THIS

Regression suite after every sprint

Why: Runs every 2 weeks — automation saves 40+ hrs/sprint

Login flow with 50 different user types

Why: Data-driven automation — 50 tests in under 2 minutes

Cross-browser checkout on 5 browsers

Why: Playwright runs all 5 browsers in parallel, simultaneously

API response validation for 200 endpoints

Why: RestAssured automates 200 checks in under 5 minutes

Performance: 10,000 concurrent users

Why: Cannot simulate manually — JMeter/k6 required

KEEP MANUAL

Testing a brand-new, unstable feature

Why: UI/flow changes daily — automation scripts break constantly

Usability review of a redesigned checkout

Why: Requires human judgement on experience quality

One-off data migration validation

Why: Runs once — automation investment not justified

UAT with business users

Why: Business users execute manually — no scripting required

Exploratory testing of new mobile app

Why: Requires adaptive investigation — not defined steps

SECTION

6

Exploratory & Ad-hoc Testing

Unscripted, intelligent testing approaches that leverage human expertise

Exploratory Testing

Simultaneous learning, designing, and executing — the tester as detective

Manual

Exploratory Testing is a simultaneous process of LEARNING about the software, DESIGNING tests on the fly, and EXECUTING them — all at the same time. Unlike scripted testing, there are no predefined steps. The tester uses their knowledge, intuition, and creativity to probe the system for unexpected behaviour.

🔍 Characteristics of Exploratory Testing

- Tester has freedom to follow intuition — no rigid test script
- Based on the tester's domain knowledge and experience
- Most effective for discovering unexpected edge cases and bugs
- Guided by a 'test charter' — a high-level goal, not a script
- Time-boxed sessions (typically 60–120 minutes)
- Observations and findings are documented during or after the session

When to Use Exploratory Testing

- After a new feature is released to QA — discover unknown risks
- When scripted testing coverage is complete — find what scripts missed
- New application with limited documentation
- Time-critical situations where full scripting is not possible
- Investigating a production bug to understand root cause
- Validating new user journeys before formal test cases are written

✅ Key Benefits

- Finds defects that scripted tests consistently miss
- Most closely simulates real user behaviour
- Adapts to what is found — follows interesting threads

Real-World Example

- Charter: 'Explore the payment flow for unusual currency inputs'
- Tester discovers: entering £ symbol causes server error (500)
- Bug not in any scripted test — found through exploration

Ad-hoc Testing

Informal, unplanned testing without documentation or predefined objectives

Manual

Ad-hoc Testing is the most informal type of testing — performed WITHOUT any test plan, test cases, or documentation. Testers access the application randomly and try to find defects based purely on instinct and experience. It requires no preparation and follows no defined process.

✅ Benefits of Ad-hoc Testing

- Requires zero preparation time — start immediately
- Can be performed at any time by any team member
- Often finds defects that formal processes miss
- Effective for quick sanity checks during development
- Useful for rapid exploration of new builds
- Good for time-critical situations with no scripting time
- Encourages creative thinking — no boundaries on test approach

⚠ Risks of Ad-hoc Testing

- No documentation — defects may be difficult to reproduce
- No systematic coverage — same areas may be tested repeatedly
- Inconsistent — depends heavily on individual tester's skill
- Results cannot be measured or compared across cycles
- Can miss critical areas that structured testing would cover
- Not repeatable — cannot verify same test was run next cycle
- Not suitable as the primary or only testing approach

📌 *Best Practice: Ad-hoc testing is a SUPPLEMENT, not a replacement for structured testing. When ad-hoc testing finds a defect, immediately document it formally*

SECTION

7

Alpha & Beta Testing

Internal and external validation before full public release

Alpha & Beta Testing

Two sequential stages of pre-release validation with different audiences

UAT/Release

Alpha Testing — Internal

- Performed INTERNALLY by the company's own team before any external release
- Conducted by QA engineers, developers, product managers, and internal staff
- Takes place in the developer's or QA's controlled environment
- Identifies major defects and usability issues before exposing to external users
- Covers functional, usability, performance, and reliability testing
- No real users involved — internal team simulates user behaviour
- Environment: Staging/UAT — close to but not equal to production
- Example: Microsoft employees test Windows internally before any external release

Beta Testing — External

- Performed EXTERNALLY by real end users in a real-world environment
- Conducted by selected customers, early adopters, or the general public
- Software is released to a limited group of actual end users
- Captures real-world usage patterns, edge cases, and feedback
- Users test in their own environments — unpredictable and diverse
- Company collects feedback, bug reports, and performance data
- Environment: Production or near-production — real traffic and data
- Example: Google releases Chrome beta to millions of public testers

Alpha & Beta Testing

Two sequential stages of pre-release validation with different audiences

UAT/Release

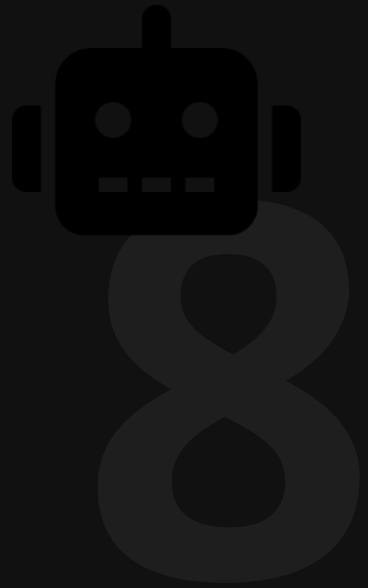
Factor	Alpha Testing	Beta Testing
Who tests	Internal team (QA, Dev, PM, Employees)	External real users / selected customers
Environment	Controlled — QA or staging environment	Real-world — production-like or production
Feedback type	Detailed technical bug reports	User experience feedback + bug reports
Control	Full control by the company	Limited control — users in own environment
Timing	Before beta release	After alpha, before full public release

SECTION

8

Test Automation in Modern QA

Selenium • Playwright • IntelliJ • Eclipse — tools and workflow



The Role of Automation in Modern Quality Engineering

Why automation is no longer optional — it is a core QA engineering discipline

Test Automation is the use of software to control the execution of tests, compare actual results to expected results, and report outcomes — without human interaction. In modern QA, automation is not a luxury — it is a fundamental requirement for sustainable quality at speed.

🚀 Why Automation is Essential

- Agile teams release every 1–2 weeks — manual regression is impossible
- CI/CD pipelines require automated tests to gate every code merge
- Scale: 500+ regression tests cannot be run manually before each release
- Consistency: scripts execute identically every time — no human fatigue
- Speed: Playwright can run 200 tests in 5 minutes across 3 browsers
- Early feedback: developers get test results within minutes of committing code
- Cost at scale: after initial investment, each test run costs near zero

⚠️ When NOT to Automate

- Features that change frequently — automation maintenance costs more than benefit
- One-time tests — scripts take longer to write than to execute manually
- Exploratory testing — requires human judgement and adaptability
- Usability and UX testing — human evaluation of experience is required
- Visual design testing — pixel-perfect comparison needs human review
- Ad-hoc investigation — informal probing doesn't suit scripted flows
- Early prototype stages — UI and flow change daily, scripts break constantly

Selenium — Web Browser Automation Framework

The industry-standard, open-source framework for automating web applications

Automation

Selenium is the world's most widely used open-source browser automation framework. It allows QA engineers to write test scripts in Java, Python, C#, or JavaScript that control real web browsers — clicking, typing, navigating, and verifying outcomes exactly as a user would.

Key Components

- Selenium WebDriver — core API for controlling browsers programmatically
- Selenium Grid — run tests in PARALLEL across multiple machines and browsers
- Selenium IDE — record-and-playback tool (for beginners/quick scripts)
- Page Object Model (POM) — design pattern for maintainable test architecture
- TestNG / JUnit — test frameworks integrated with Selenium for Java

IDE Integration

- IntelliJ IDEA: Most popular at Lubanzi — excellent Maven/Gradle support
- Eclipse IDE: Free, widely used in Java/Selenium enterprise teams
- Maven/Gradle: Build tools that manage Selenium JAR dependencies
- TestNG plugin: IntelliJ/Eclipse — run and report individual test methods
- GitHub Actions / Jenkins: Run Selenium Grid tests in CI/CD pipeline

Supported Browsers

- Chrome (ChromeDriver) • Firefox (GeckoDriver)
- Edge (EdgeDriver) • Safari (SafariDriver)
- Selenium Grid: Run all 4 browsers simultaneously in parallel

Typical Project Setup (Java)

- 1. IntelliJ + Maven project 2. Add selenium-java dependency
- 3. Add TestNG 4. Create Page Objects 5. Write @Test methods
- 6. Run via TestNG XML suite 7. View ExtentReports HTML output

Playwright — Modern Cross-Browser Automation

Microsoft's fast, reliable, feature-rich automation framework for modern web apps

Automation

Playwright is Microsoft's modern, open-source browser automation framework designed to address the limitations of Selenium. It offers faster execution, built-in auto-wait, native cross-browser support, and rich developer tooling — making it the preferred choice for new QA automation projects.

Key Advantages Over Selenium

- Auto-wait: automatically waits for elements — far fewer flaky tests
- Native cross-browser: ONE script runs Chromium, Firefox, and WebKit
- Built-in tracing: full test replay with screenshots, network, DOM snapshots
- Faster execution: parallelism built in — no Grid configuration needed
- API testing: supports HTTP requests alongside UI tests in same script
- Codegen: record browser interactions and auto-generate test code

IDE & Language Support

- IntelliJ IDEA: Java + Playwright (com.microsoft.playwright dependency)
- VS Code: TypeScript/JavaScript (most common for Playwright)
- PyCharm: Python + Playwright (playwright.sync_api)
- All IDEs: Playwright test runner with built-in reporter
- IntelliJ + Maven: same setup pattern as Selenium — familiar to Java teams

PLAYWRIGHT vs SELENIUM — QUICK REFERENCE

Feature	Playwright	Selenium
Auto-wait	✅ Built-in, intelligent	❌ Manual waits/sleeps required
Cross-browser	✅ Chromium + Firefox + WebKit	⚠ Requires separate WebDrivers

Test Automation Workflow — End to End

How a complete automated testing pipeline works in a real project

Identify Automation Candidates

Analyse test suite: high frequency + stable + repeatable = automate. Prioritise regression suite and critical user journeys. Output: Automation backlog.

Set Up Framework

Create project in IntelliJ (Maven + Java). Add dependencies: selenium-java / playwright. Set up Page Object Model structure. Configure TestNG or JUnit runner. Connect to CI/CD.

Write Test Scripts

Write Page Objects for each UI screen. Write test methods using @Test annotations. Add assertions: assertEquals, assertTrue. Parameterise with data providers (data-driven testing).

Execute Locally

Run tests in IntelliJ via TestNG XML suite. Verify all tests pass on local environment. Debug and fix any script issues. Review Playwright trace viewer or ExtentReports.

Integrate with CI/CD

Push scripts to Git repository. Configure Jenkins / GitHub Actions pipeline. Pipeline: code commit → run automation → report results → notify team. Fail pipeline on test failure.

Monitor & Maintain

Review results after every run. Fix flaky tests immediately. Update scripts when UI changes. Expand suite with new test cases each sprint. Track automation coverage %.

SECTION

9

Real-World Testing Workflow

How ALL testing types come together in a real software project

Real-World Project: E-Commerce Platform Launch

Step-by-step: how every testing type is applied from development to production

Sprint 1–2 **Development**	**Tests: Unit Testing • Integration Testing (API layer)** Who: Developers (unit) + QA (integration) *Tools: JUnit/TestNG (unit) • Postman (API)*	Individual components validated in isolation
Sprint 3 **QA Build Arrives**	**Tests: Smoke Testing • Exploratory Testing** Who: QA Engineers *Tools: Playwright CI smoke suite • Manual exploration*	Build confirmed stable. Unknown risks surfaced.
Sprint 4 **Full QA Cycle**	**Tests: System Testing • Regression Testing • Performance Testing • Security Testing** Who: QA Engineering Team *Tools: Selenium/Playwright (system) • JMeter (perf) • ZAP (security)*	Full functional coverage. Non-functional baseline established.
Sprint 5 **Pre-Release**	**Tests: UAT • Compatibility Testing • Sanity Testing (after fixes)** Who: Business Users (UAT) • QA (compatibility/sanity) *Tools: Playwright cross-browser • Manual UAT sessions*	Business sign-off. Cross-browser compatibility confirmed.
Release Day **Production Deploy**	**Tests: Smoke Testing (production) • Alpha Testing** Who: QA + DevOps *Tools: Playwright smoke suite triggered by deployment pipeline*	Production health confirmed. Go/No-Go decision made.
Post-Release **Live**	**Tests: Beta Testing • Monitoring • Regression (next sprint)** Who: External beta users + QA *Tools: Hotjar (UX) • Datadog (monitoring) • Playwright CI regression*	Real-world validation. Continuous regression safety net active.

SECTION

10

Best Practices & Summary

Key principles for effective testing + complete recap of all testing types

Best Practices in Software Testing — Part 1

Principles that separate good testers from great testers

📐 Write Tests for Requirements, Not Code

Every test case must trace back to a specific requirement. If you can't point to a requirement, the test case's value is questionable. Use the RTM to maintain traceability.

🔺 Follow the Testing Pyramid

70% unit tests (fast, cheap), 20% integration tests, 10% UI/E2E tests (slow, expensive). Inverting the pyramid creates slow, expensive, fragile test suites.

⬅️ Shift Testing Left

Get QA involved from the requirements phase. Review specifications before development begins. Find defects in documents — not in production code.

⚠️ Risk-Based Test Prioritisation

Not all features are equal. Test the most critical, highest-risk features most thoroughly. Allocate 80% of testing effort to 20% of highest-risk functionality.

🔄 Automate Regression Continuously

Add new regression tests every sprint. Run the full regression suite on every merge via CI/CD. Never let the regression suite shrink — quality debt compounds.

🐛 Log Every Defect Formally

Every deviation from expected behaviour must be formally documented with steps to reproduce, environment, severity, and priority. Verbal bug reports are lost bugs.

Best Practices in Software Testing — Part 2

Advanced principles for continuous quality improvement

🤝 Collaborate Early with Developers

QA is not the gatekeeper who rejects developer work. QA is a quality partner who collaborates throughout. Pair with developers during development — not after. Shared ownership of quality produces better outcomes.

📊 Measure and Track Quality Metrics

Track: defect density, test coverage, pass/fail rate, defect leakage (escaped to production), automation coverage %. Use metrics to improve — not to blame. Data-driven QA improves continuously.

🔄 Apply Continuous Testing in CI/CD

Testing must run automatically on every code change — not just at the end of a sprint. Playwright and Selenium suites integrated with Jenkins/GitHub Actions provide real-time quality feedback to developers.

🧠 Invest in Exploratory Testing

Allocate 20% of testing time to unscripted exploration. Scripted tests validate known scenarios. Exploratory testing discovers unknown risks. Both are necessary for comprehensive coverage.

📚 Maintain Test Documentation

Test cases, plans, and results must be documented and updated. Undocumented tests cannot be reproduced, reviewed, or improved. Test documentation is a team asset — not a personal record.

🚀 Retrospect and Improve Every Cycle

After every release: review defect escape rate, test coverage gaps, and estimation accuracy. Use Lessons Learned to update processes. Great QA teams get measurably better each cycle.

Summary — All Testing Types at a Glance

Your complete reference card: every testing type covered in this session

Testing Type	Category	Purpose	Key Tools	When Used
Unit Testing	Functional/Auto	Test individual code functions	JUnit, TestNG, Mockito	During development (TDD)
Integration Testing	Functional	Test component interactions	Postman, Selenium, WireMock	After unit testing
System Testing	Functional	End-to-end validation	Selenium, Playwright	Before UAT
UAT	Functional	Business acceptance	Manual, test management tools	Before release, by business
Smoke Testing	Functional	Build stability check	Playwright CI, Postman	After every deployment
Sanity Testing	Functional	Verify specific fix	Manual + targeted scripts	After bug fix
Regression Testing	Functional/Auto	Prevent old bugs returning	Selenium, Playwright + CI	Before every release
Load Testing	Non-Functional	Normal traffic performance	JMeter, k6, Gatling	Before release/major events
Stress Testing	Non-Functional	Find breaking point	JMeter, Locust	Capacity planning
Spike Testing	Non-Functional	Handle sudden traffic surge	Gatling, k6	Before viral campaigns
Endurance Testing	Non-Functional	Long-term stability	JMeter (long runs)	Before 24/7 live systems
Security Testing	Non-Functional	Find vulnerabilities	OWASP ZAP, Burp Suite	Before every release
Usability Testing	Non-Functional	UX quality	Hotjar, UserTesting.com	Design & release phases

Key Takeaways

The most important concepts from this training session

Testing is a discipline, not a phase:
Quality engineering runs throughout the entire SDLC — from requirements to post-production monitoring. Shift-left means finding bugs earlier, where they are cheapest to fix.

Know your categories:
Every test type belongs to a category: Functional/Non-Functional, Manual/Automated. Understanding the category tells you why the test exists and how to approach it.

Functional + Non-Functional = Complete testing:
Functional testing confirms the app works. Non-functional testing confirms it performs, scales, and is secure. You need BOTH — one without the other leaves critical gaps.

Automate what is stable and repetitive:
Regression suites, smoke tests, and CI/CD gates are perfect for Selenium and Playwright automation. Exploratory, usability, and one-off tests stay manual.

Selenium & Playwright are your core tools:
Selenium (Java/IntelliJ/Eclipse) for established projects. Playwright for modern, fast, cross-browser automation. Both are industry standards at Lubanzi.

Every testing type has a purpose:
No testing type is redundant. Smoke tests protect your execution time. Regression protects existing quality. Security testing protects your users. Apply each at the right moment.

Measure, learn, and improve:
Track defect density, test coverage, and automation percentage. Use metrics to continuously improve your QA practice. Data-driven QA engineering is the mark of a senior professional.